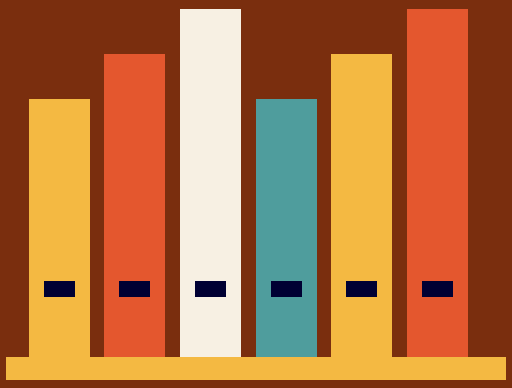

Essential Programming Libraries

How to choose, use and publish libraries in JavaScript, Python and .NET

Choose • Install • Use • Maintain

Updated October 2026

Contents

1. Libraries, frameworks and packages
2. How to choose a library
3. Installing and managing dependencies
4. JavaScript and TypeScript libraries
5. Python libraries
6. .NET (C#) libraries
7. Libraries in other languages
8. Building and publishing your own
9. Checklist and common problems

Why libraries matter
Libraries let you reuse tested code for common problems such as dates, HTTP, data analysis and logging, so you can focus on what makes your application unique. The skill is choosing well and depending on as little as you need.

What this guide covers

- A simple method for evaluating any library before you adopt it.
- Commands for installing and updating packages in JavaScript, Python and .NET.
- Well-known libraries by category, with short working examples.
- How to publish your own library for others to use.

Note on accuracy
Library popularity, licences and maintenance status change over time. Treat the lists as starting points and check each project's page before relying on it.

1. Libraries, frameworks and packages

These terms are often mixed up. The key difference is **who is in control**.

Term	Meaning	Example
Library	A collection of code **you call** when you need it.	date-fns, requests, Serilog
Framework	A structure that **calls your code** and defines how the app is organized.	Angular, Django, ASP.NET Core
Package	A distributable unit (library, tool or framework) published to a registry.	Anything on npm, PyPI or NuGet
Dependency	A package your project needs in order to work.	Listed in package.json, requirements.txt, .csproj
Transitive dependency	A dependency of one of your dependencies.	Installed automatically, often hundreds

Package registries

Ecosystem	Registry	Tool	Manifest file
JavaScript / TypeScript	npm	npm, pnpm, yarn	`package.json`
Python	PyPI	pip, uv, Poetry	`pyproject.toml`, `requirements.txt`
C# / .NET	NuGet	dotnet CLI, Visual Studio	`*.csproj`
Java	Maven Central	Maven, Gradle	`pom.xml`, `build.gradle`
Rust	crates.io	Cargo	`Cargo.toml`
Go	Go module proxy	go	`go.mod`

> **Every dependency is a trade-off**
> Each library saves you time but adds code you did not write: more to update, more that can break, and more that can be attacked. Prefer a few well-chosen libraries over many small ones.

2. How to choose a library

Before adding a dependency, spend five minutes evaluating it. Use this checklist:

Question	Where to look	Good sign
Is it maintained?	Latest release date, recent commits	Releases within the last year; issues get replies
Is it popular and trusted?	Weekly downloads, GitHub stars, who uses it	Widely used, with active community
Is the licence compatible?	LICENSE file, registry page	MIT, Apache 2.0, BSD for most projects
Is it secure?	Security advisories, `npm audit`, Snyk	No unpatched known vulnerabilities
Is it well documented?	README, docs site, examples	Clear quick start and API reference
Is it the right size?	Bundlephobia (JS), install size	Small, few dependencies of its own
Does it fit your stack?	Supported versions, TypeScript types	Works with your runtime and tools
Is it stable?	Version number, changelog, deprecation notes	Follows semantic versioning; few breaking changes

Questions to ask yourself

- Can I solve this with the **standard library** or a few lines of code? Modern languages include a lot.
- Is this a **core** part of my product? If so, a dependency may be risky; if not, buying in is wise.
- What happens if the library is **abandoned**? Is there an alternative or an easy way to replace it?
- Will this library be **hard to remove** later? Wrap it behind your own small interface if it is deeply used.

> **Warning signs**
> No releases for several years, many unanswered issues, a single maintainer with no succession plan, a name that is one letter different from a famous package (**typosquatting**), or a request for unusual permissions.

3. Installing and managing dependencies

Installing commands

```
# JavaScript / TypeScript (npm)
npm install date-fns             # runtime dependency
npm install -D vitest            # development-only dependency
npm update                       # update within allowed ranges
npm uninstall date-fns

# Python (pip, inside a virtual environment)
python -m venv .venv
pip install requests pandas
pip freeze > requirements.txt    # record exact versions
pip install -r requirements.txt

# .NET (NuGet)
dotnet add package Serilog
dotnet list package --outdated
dotnet remove package Serilog
```

Understanding version numbers

Most libraries use **Semantic Versioning**: `MAJOR.MINOR.PATCH` (for example 4.2.1).

- **PATCH** (4.2.**1**): bug fixes, safe to update.
- **MINOR** (4.**2**.1): new backward-compatible features.
- **MAJOR** (**4**.2.1): breaking changes; read the migration guide first.

In npm, `^4.2.1` allows minor and patch updates; `~4.2.1` allows only patch updates; an exact version `4.2.1` never changes.

Good dependency habits

- **Commit your lockfile** (`package-lock.json`, `poetry.lock`, `uv.lock`) so every machine installs identical versions.
- **Isolate environments**: use virtual environments in Python and per-project installs in Node.
- **Update regularly** in small steps, using Dependabot or Renovate to open pull requests automatically.
- **Audit**: run `npm audit`, `pip-audit` or `dotnet list package --vulnerable`.
- **Remove unused packages** to reduce size and attack surface.
- **Check licences** of all dependencies, especially in commercial products.

4. JavaScript and TypeScript libraries

Category	Popular libraries	Notes
UI	React, Vue, Svelte, Solid	Component libraries for building interfaces.
State management	Zustand, Redux Toolkit, Pinia (Vue), Jotai	Share data between components.
Data fetching	TanStack Query, SWR, Axios, native `fetch`	Caching, retries and loading states.
Forms and validation	React Hook Form, Zod, Yup, Valibot	Zod also infers TypeScript types.
Dates and times	date-fns, Day.js, Luxon	Avoid heavy older libraries when possible.
Utilities	Lodash (or lodash-es), Ramda	Many helpers now exist natively in JavaScript.
Charts	Chart.js, D3, Recharts, ECharts	D3 is the most powerful, Chart.js the simplest.
Animation	Framer Motion, GSAP, Motion One	Transitions and interactive effects.
UI components	shadcn/ui, Material UI, Chakra UI, Radix	Accessible ready-made components.
Testing	Vitest, Jest, Testing Library, Playwright	Unit, component and end-to-end tests.
Server (Node.js)	Express, Fastify, Hono, Prisma, Drizzle	Back-end APIs and database access.

Example: date-fns and Zod

```
import { format, addDays } from "date-fns";
import { z } from "zod";

console.log(format(addDays(new Date(), 7), "yyyy-MM-dd"));

const User = z.object({
  name: z.string().min(1),
  age: z.number().int().positive(),
});

const result = User.safeParse({ name: "Ada", age: 36 });
if (!result.success) {
  console.error(result.error.issues);
} else {
  console.log("Valid user:", result.data);
}
```

Tip: check bundle size
In frontend projects, every kB is downloaded by users. Look up packages on **bundlephobia.com** and prefer tree-shakeable libraries that let you import only what you use.

5. Python libraries

Category	Popular libraries	Notes
Data analysis	NumPy, pandas, Polars	NumPy for arrays, pandas for tables, Polars for fast large data.
Visualization	Matplotlib, Seaborn, Plotly, Altair	Plotly produces interactive charts.
Machine learning	scikit-learn, PyTorch, TensorFlow, XGBoost	scikit-learn is the classic starting point.
Web frameworks	FastAPI, Flask, Django	FastAPI for APIs, Django for full sites.
HTTP clients	requests, httpx, aiohttp	httpx supports async.
Web scraping	Beautiful Soup, Scrapy, Playwright	Respect site terms and robots.txt.
Data validation	Pydantic, marshmallow	Pydantic powers FastAPI.
Command-line tools	Typer, Click, rich, argparse	rich adds colors and tables.
Databases	SQLAlchemy, psycopg, sqlite3 (built in)	SQLAlchemy is a popular ORM.
Desktop GUI	Tkinter (built in), PySide6, PyQt, Kivy	See the Windows desktop guide for context.
Testing	pytest, hypothesis, unittest.mock	pytest is the de facto standard.
Code quality	Ruff, Black, mypy	Linting, formatting and type checking.

Examples

```
# requests: call an API
import requests

r = requests.get("https://api.github.com", timeout=10)
r.raise_for_status()
print(r.json())

# pandas: total revenue per region from a CSV
import pandas as pd

df = pd.read_csv("sales.csv")
print(df.groupby("region")["revenue"].sum())

# FastAPI: a tiny web API (run: uvicorn main:app --reload)
from fastapi import FastAPI

app = FastAPI()

@app.get("/hello/{name}")
def hello(name: str):
    return {"message": f"Hello, {name}"}
```

6. .NET (C#) libraries

Most .NET libraries are NuGet packages. Many are installed with `dotnet add package` or the NuGet Package Manager in Visual Studio.

Category	Popular libraries	Notes
Database access	Entity Framework Core, Dapper	EF Core is a full ORM; Dapper is a fast micro-ORM.
JSON	System.Text.Json (built in), Newtonsoft.Json	System.Text.Json is the default in modern .NET.
Logging	Serilog, NLog, Microsoft.Extensions.Logging	Serilog offers structured logging with many sinks.
Resilience	Polly	Retries, timeouts and circuit breakers.
Testing	xUnit, NUnit, NSubstitute, FluentAssertions	See licence terms of each before adopting.
WPF / MVVM	CommunityToolkit.Mvvm, MaterialDesignInXAML	Helpers and styles for WPF desktop apps.
Windows Forms	ScottPlot, LiveCharts2	Charts for desktop apps.
HTTP / APIs	HttpClient (built in), Refit, RestSharp	Refit generates clients from interfaces.
Data / Excel	CsvHelper, ClosedXML, EPPlus	Check EPPlus licensing for commercial use.
Web	ASP.NET Core, Swashbuckle / Scalar (OpenAPI docs)	Built-in framework plus API documentation.

Example: add Serilog to a console app

```
dotnet add package Serilog
dotnet add package Serilog.Sinks.Console
```

```
using Serilog;

Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .WriteTo.Console()
    .CreateLogger();

Log.Information("Application started at {Time}", DateTime.Now);
Log.Warning("Disk space is low: {FreeMb} MB", 120);
Log.CloseAndFlush();
```

Licences change
Several popular .NET libraries have moved from free open-source licences to commercial or dual licences in recent years. Always read the current licence before adding a package to a commercial project.

7. Libraries in other languages

Language	Notable libraries	Typical use
Java / Kotlin	Spring Boot, Jackson, JUnit, Hibernate, Mockito, Ktor	Enterprise back ends, Android
Go	Gin, Echo, Cobra, GORM, testify	APIs, command-line tools
Rust	Tokio, Serde, Clap, Axum, Reqwest	Performance-critical and system software
PHP	Laravel, Symfony, Guzzle, PHPUnit	Websites and CMS back ends
Ruby	Rails, RSpec, Sidekiq	Web applications
Swift	SwiftUI, Alamofire, Combine	iOS and macOS apps
Dart	Flutter, Riverpod, Dio	Cross-platform mobile apps

Cross-language building blocks

Some libraries solve the same problem in every language:

- **JSON, YAML and Protocol Buffers** libraries for data exchange.
- **HTTP clients** for calling APIs.
- **Logging** libraries with levels and structured output.
- **Date and time** libraries to handle time zones correctly.
- **Test frameworks and mocking** libraries.
- **Cryptography** libraries, such as OpenSSL bindings, libsodium and your language's built-in crypto module.

> **Never write your own crypto**
> Use a well-reviewed library for encryption, hashing and password storage (for example bcrypt, Argon2 or the built-in APIs). Home-made cryptography is almost always insecure.

8. Building and publishing your own

If you write code that others could reuse, turn it into a library. Even internal libraries help teams avoid copy-and-paste.

Design principles

- Solve **one problem** well, with a small, clear public API.
- Keep **dependencies minimal**; every one you add is inherited by your users.
- Hide internals; only export what users need. Changes to the public API are breaking changes.
- Write **tests**, **documentation** and examples, and a README that shows a working quick start.
- Follow **semantic versioning** and keep a CHANGELOG.
- Pick a **licence** (MIT and Apache 2.0 are common permissive choices).

Typical project layout

```
my-library/
  src/                  # source code
  tests/                # automated tests
  docs/                 # documentation
  examples/             # small runnable examples
  README.md
  LICENSE
  CHANGELOG.md
  package.json | pyproject.toml | MyLibrary.csproj
```

Publishing commands

Ecosystem	Steps
npm	Set name and version in `package.json`, then `npm login` and `npm publish`.
PyPI	Configure `pyproject.toml`, then `python -m build` and `twine upload dist/*`.
NuGet	Set package metadata in the .csproj, then `dotnet pack -c Release` and `dotnet nuget push`.

Automate releases

Use CI (for example GitHub Actions) to run tests on every pull request and to publish a new version automatically when you create a tagged release. Use **trusted publishing** or short-lived tokens rather than long-lived passwords, and protect your registry account with two-factor authentication.

9. Checklist and common problems

Before adding a library

- [ ] I checked whether the standard library can do this
- [ ] The project is actively maintained and has recent releases
- [ ] The licence fits my project (including commercial use)
- [ ] No unresolved security advisories
- [ ] Documentation and examples are clear
- [ ] The size and number of its own dependencies are reasonable
- [ ] I know how I would replace it if it were abandoned

Common problems and fixes

Problem	What to do
"Works on my machine"	Commit the lockfile, pin runtime versions, and use a clean install (`npm ci`) in CI.
Dependency conflicts	Check which packages need different versions; update them together or choose compatible versions.
Vulnerability alerts	Update the package; if no fix exists, look for an alternative or an override and document the risk.
Huge install or bundle size	Remove unused packages, import only what you need, and replace heavy libraries with lighter ones.
Breaking change after update	Roll back using Git, read the changelog, and update in a separate branch.
Abandoned dependency	Fork it, replace it, or wrap it behind your own interface for easy swapping.
Supply-chain attack worry	Use lockfiles, review new dependencies, enable audit tools, and avoid packages with suspicious names.

> **Final advice**
> A good library is one you barely have to think about. Choose carefully, keep it updated, and keep your own code decoupled so that swapping a library is a small change, not a rewrite.